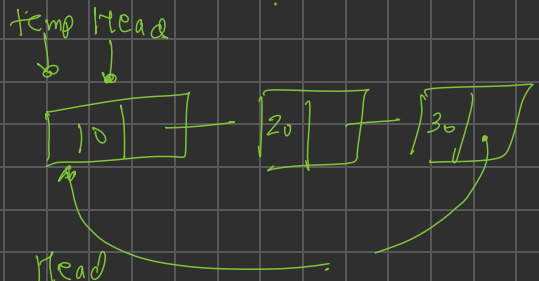
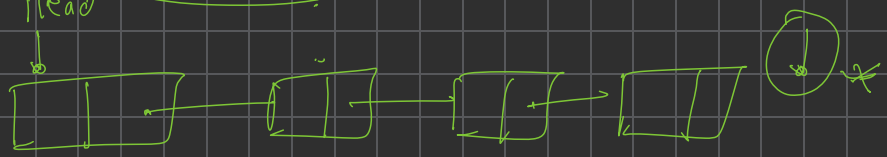
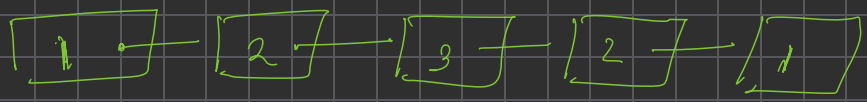
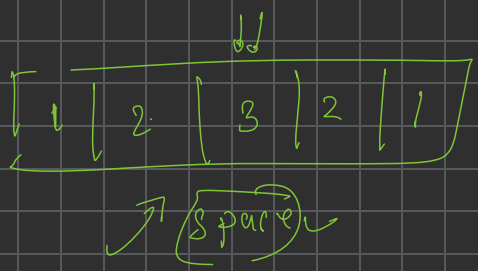


Circular linked



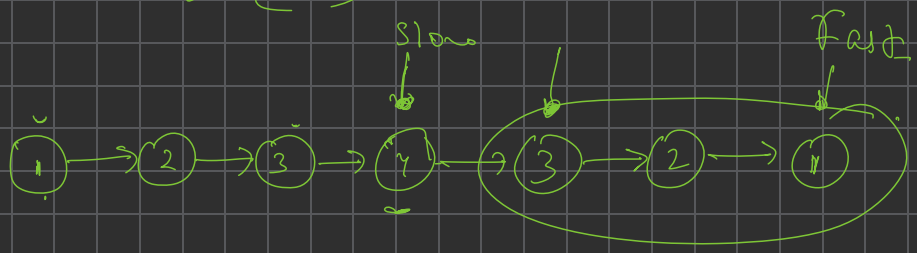


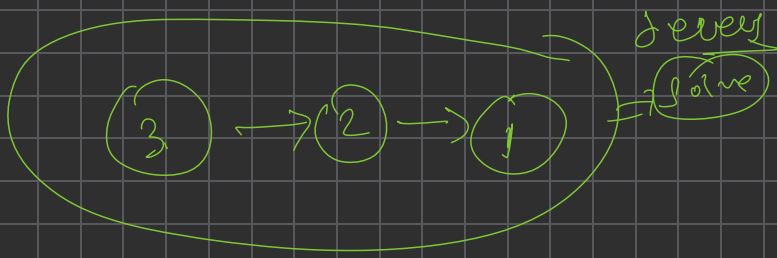
O(n)
+ O(n)



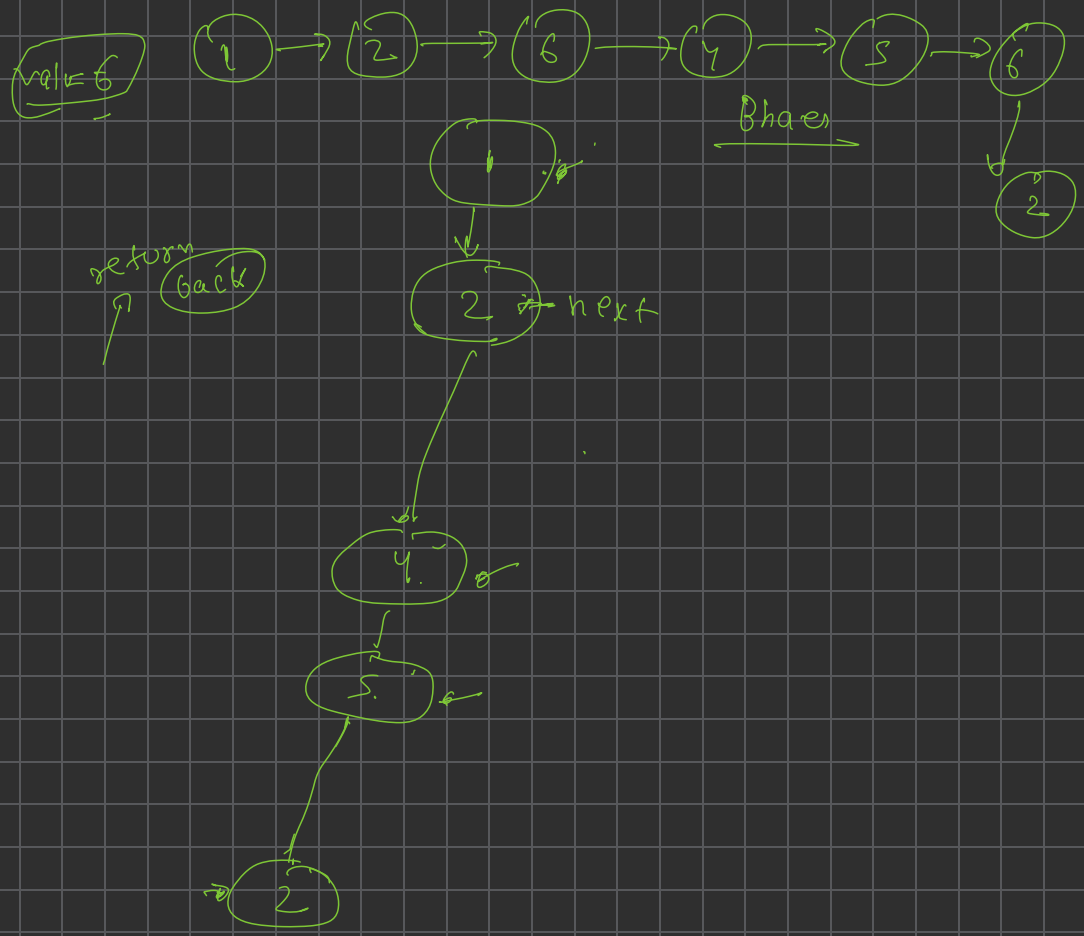
Palindrome

O(n)





every
solve



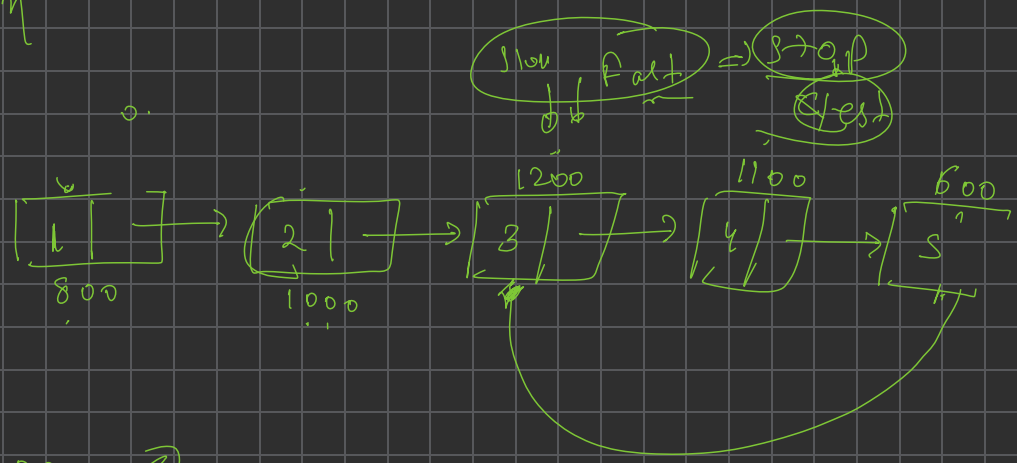

```
ListNode* Delete( ListNode * head, int value)
{
    if ( !head)
        retur head;

head->next = Delete ( head -> next, value)

    if (head -> val == value)
    {
        ListNode* temp = head->next;
        delete head;
```

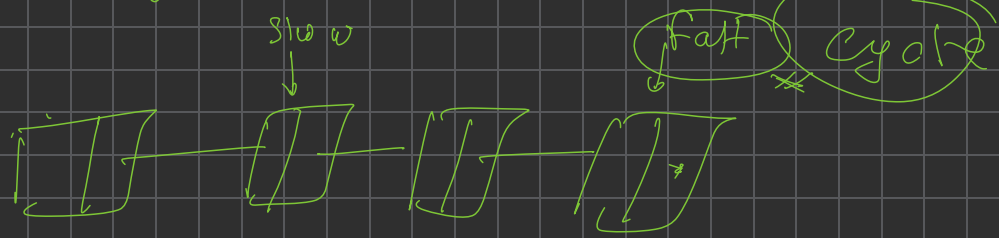
```
        return temp;
    }

    return head;
}
```

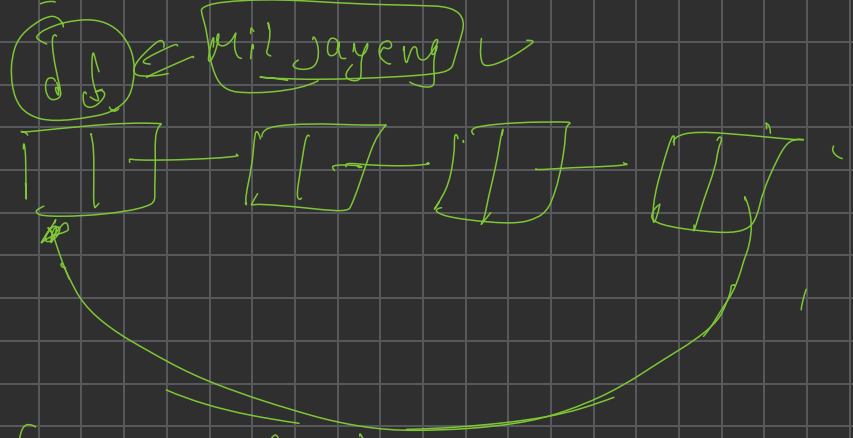


800
1000
1200
1100
600

Extra space

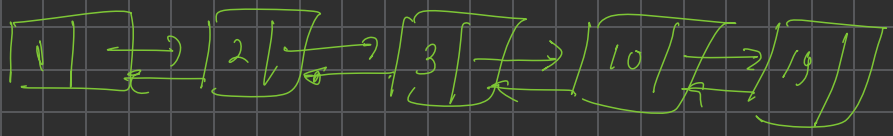


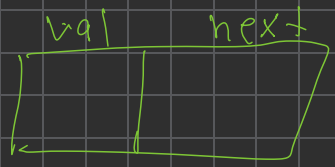

```
slow = head, fast = head -> next;
while(fast && fast -> next && slow != fast)
{
    fast = fast -> next -> next;
    slow = slow -> next;
}
```



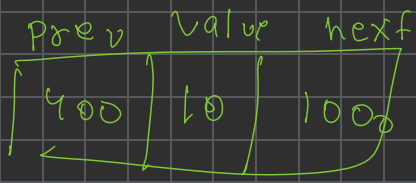

```
if (slow == fast)
    return 1;
return 0;
```

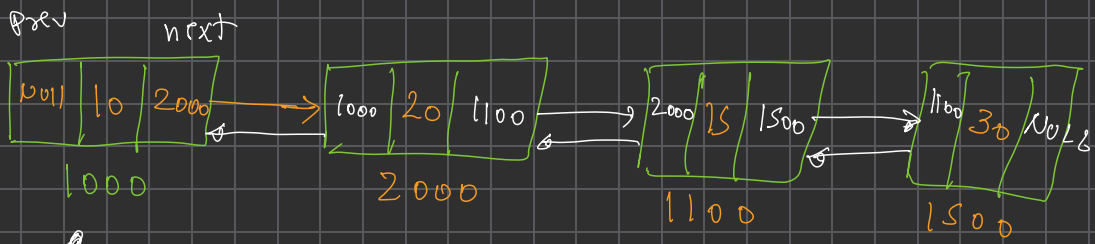
{Doubly Linked List}





Single





Head

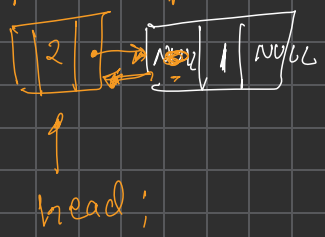
```
Class Node
{
    Public :
    int value;
    Node * prev, * next

    Node (int val)
    {
        value = val;
        prev = next = NULL;
    }
```

Insertion ?

→ Start

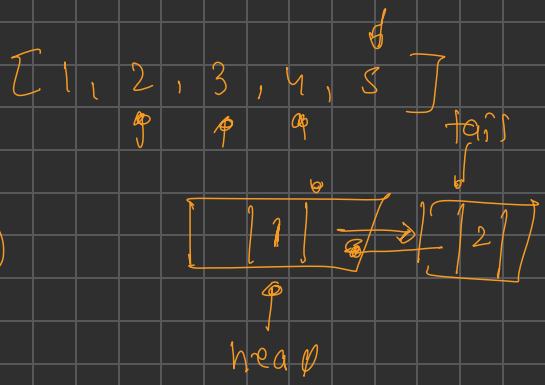
→ End

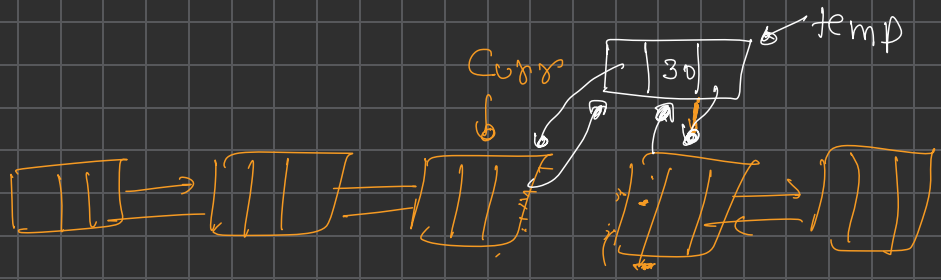



```
Node * head = NULL;
int arr[5] = {1, 2, 3, 4, 5}
for (i=0; i<5; i++
{
    if (!head)
    {
        head = new Node(arr[0]);
    }
    else
    {
        Node * temp = new Node(arr[i]);
        temp -> next = head;
        head -> prev = temp;
        head = temp;
    }
}
```

```
Node * head = NULL,
       * tail = NULL;

for (i=0; i<5; i++)
{
    if (! head)
    {
        head = new Node(arr[i]);
        tail = head;
    }
    else
    {
        Node * temp = new Node(arr[i]);
        tail -> next = temp
        temp -> prev = tail
        tail = temp;
    }
}
```

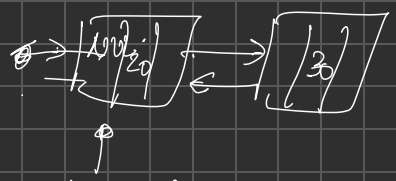




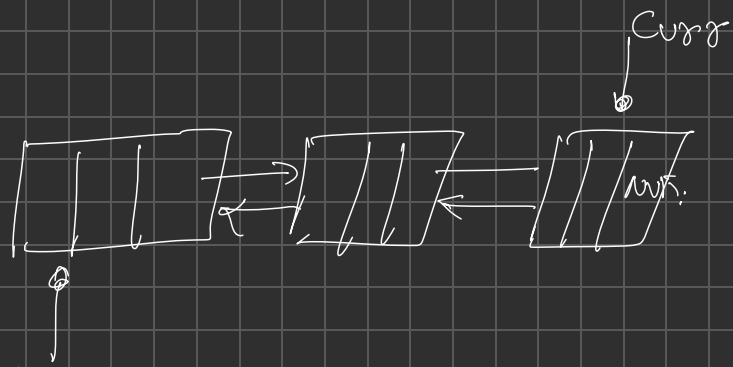

```
Node *temp = new Node(30);
temp -> next = curr -> next
temp -> prev = curr;
curr -> next -> prev = temp
curr -> next = temp;
```

(*) Deletion

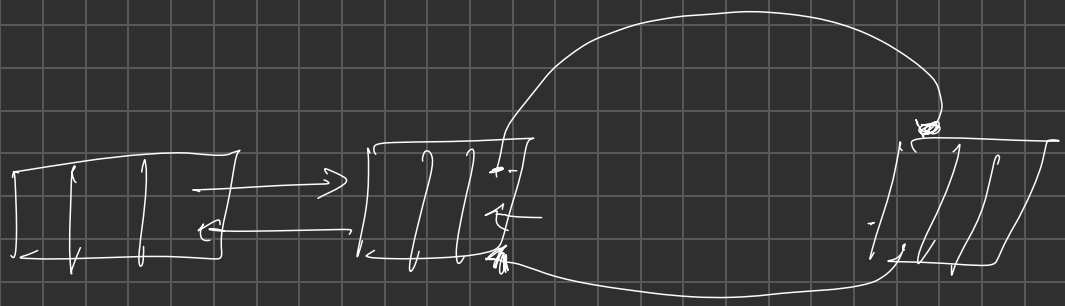
[First node
Last node
Middle]




```
head = head -> next
delete head -> prev
head -> prev = NULL;
```




```
delete curr->next;
curr->next=NULL;
```



```
curr -> prev -> next = curr -> next;
curr -> next -> prev = curr -> prev;
delete curr;
```